Programación de Procesos y Servicios
UD 1 – Concurrencia
Ejercicios 4

Programación Multihilo y sincronización

Ejercicio 1 — Hilos escritores y sincronización

Crea un programa Java que trabaje con 3 hilos.

El hilo principal deberá crear y lanzar dos hilos:

El primer hilo deberá escribir por consola la palabra "Hola" 10 veces, esperando 1 segundo entre cada escritura.

El segundo hilo deberá escribir por consola la palabra "DAM" 10 veces, esperando 1 segundo entre cada escritura.

El hilo principal deberá lanzar primero el hilo que escribe "Hola" y, después de 50 ms, lanzar el hilo que escribe "DAM".

Cada mensaje deberá aparecer acompañado del nombre del hilo que lo ha generado.

Por ejemplo:

Hilo-Hola: Hola

Hilo-DAM: DAM

Hilo-Hola: Hola

Hilo-DAM: DAM

Una vez lanzados los dos hilos, el hilo principal deberá esperar a que ambos finalicen utilizando join().

Después de 5 segundos desde el comienzo de la ejecución de los dos hilos, el hilo principal deberá interrumpir ambos hilos.

Cuando un hilo reciba la interrupción deberá:

- Mostrar un mensaje indicando que ha sido interrumpido.
- Finalizar su ejecución.

Finalmente, el hilo principal deberá mostrar:

Programa finalizado

Ejercicio 2 — Hilo principal esperando a un hilo secundario

Crea una aplicación Java formada por un hilo principal y un hilo secundario.

El hilo secundario deberá mostrar los siguientes mensajes:

Inicio del programa

Procesos

Servicios

Multihilo

Sincronización

Fin del programa

Entre cada mensaje deberá esperar 3 segundos.

El hilo principal deberá lanzar el hilo secundario y quedarse esperando su finalización.

Mientras está esperando, el hilo principal deberá mostrar cada segundo:

Hilo principal esperando...

El hilo principal podrá esperar como máximo el número de segundos indicado mediante un parámetro recibido por main.

Por ejemplo:

java Ejercicio2 7

En este caso, el hilo principal esperará como máximo 7 segundos.

Si el hilo secundario todavía no ha terminado después de ese tiempo, el hilo principal deberá:

- Interrumpir al hilo secundario.
- El hilo secundario deberá detectar la interrupción.
- Mostrar los mensajes que todavía no haya mostrado sin realizar esperas entre ellos.
- Finalizar.
- El hilo principal deberá esperar finalmente a que termine el hilo secundario.
- Mostrar el tiempo total de ejecución del programa.

Todos los mensajes deberán indicar qué hilo los está mostrando.

Por ejemplo:

[main] Hilo principal esperando...

[Hilo-Secundario] Procesos

[main] Hilo principal esperando...

Ejercicio 3 — Cuenta bancaria con varios hilos

Crea una clase llamada Cuenta que tenga un atributo privado:

```
private double saldo;
```

El constructor recibirá el saldo inicial.

La clase deberá disponer de:

- Un método getSaldo().
- Un método para ingresar dinero.
- Un método para retirar dinero.

Los métodos de ingreso y retirada deberán incluir un sleep() aleatorio de entre 100 y 500 ms para simular que las operaciones tardan un tiempo.

Cada operación deberá mostrar:

- Nombre del hilo.
- Tipo de operación.
- Cantidad.
- Saldo anterior.
- Saldo posterior.

Por ejemplo:

```
[Ingresador-1] Ingreso: 100 €
Saldo anterior: 500 €
Saldo posterior: 600 €
```

Crea una clase que implemente Runnable.

Cada objeto Runnable deberá realizar varias operaciones sobre la misma cuenta.

Desde main:

1. Crea una cuenta con 1000 €.
2. Crea 5 hilos que compartan la misma cuenta.
3. Algunos hilos realizarán ingresos.
4. Otros realizarán retiradas.
5. Cada hilo deberá realizar 5 operaciones.

6. Asigna un nombre diferente a cada hilo.
7. Lanza todos los hilos.
8. Utiliza join() para esperar a que todos finalicen.
9. Muestra el saldo final.

Después realiza una segunda versión del programa utilizando synchronized en los métodos que modifican el saldo.

Comprueba y explica qué diferencias existen entre ambas versiones.

Ejercicio 4 — Productor y consumidor

Crea una aplicación Java que simule una pequeña cola de trabajos utilizando varios hilos.

Crea una clase llamada ColaTrabajos que almacene números enteros.

La cola tendrá una capacidad máxima de 5 elementos.

El programa deberá tener:

- 2 hilos productores.
- 2 hilos consumidores.

Cada productor deberá generar 10 números y añadirlos a la cola.

Cada consumidor deberá extraer números de la cola y mostrarlos por consola.

Los productores deberán esperar 500 ms entre cada elemento producido.

Los consumidores deberán esperar 800 ms entre cada elemento consumido.

Los métodos para añadir y extraer elementos deberán estar correctamente sincronizados para evitar problemas de concurrencia.

Cuando la cola esté llena, los productores deberán esperar.

Cuando la cola esté vacía, los consumidores deberán esperar.

Utiliza los mecanismos de sincronización de Java necesarios para resolver el problema.

Cada mensaje deberá indicar el nombre del hilo:

[Productor-1] Produce: 25

[Productor-2] Produce: 67

[Consumidor-1] Consume: 25

[Consumidor-2] Consume: 67

El programa deberá finalizar cuando todos los productores hayan terminado y todos los elementos producidos hayan sido consumidos.

Ejercicio 5 — Carrera de hilos y recurso compartido

Crea una aplicación Java que simule una carrera entre 4 corredores.

Cada corredor será un hilo independiente.

Cada corredor deberá recorrer una distancia de 20 metros.

En cada iteración, el corredor avanzará entre 1 y 3 metros de forma aleatoria.

Después de cada avance deberá realizar un:

sleep()

de entre 200 y 600 ms.

Cada vez que avance deberá mostrar:

[Corredor-1] Ha avanzado 2 metros. Distancia: 8/20

Cuando un corredor llegue a los 20 metros deberá acceder a un objeto compartido llamado Clasificacion.

La clase Clasificacion tendrá que guardar la posición de llegada de cada corredor.

El acceso a la clasificación deberá estar protegido mediante synchronized, de forma que dos corredores no puedan obtener simultáneamente la misma posición.

Por ejemplo:

[Corredor-3] ¡Ha terminado! Posición: 1

[Corredor-1] ¡Ha terminado! Posición: 2

[Corredor-4] ¡Ha terminado! Posición: 3

[Corredor-2] ¡Ha terminado! Posición: 4

El hilo principal deberá:

- Crear la clasificación.
- Crear los 4 corredores.
- Asignar un nombre a cada hilo.
- Lanzar los 4 hilos.
- Esperar mediante join() a que todos terminen.
- Mostrar la clasificación final.
- Mostrar el tiempo total de la carrera.
- Modificación

Una vez funcionando, elimina el synchronized del método que asigna la posición y ejecuta varias veces el programa.

Explica qué problemas pueden aparecer y por qué.